clase 2 - Generación de datos

Joselina Davyt-Colo

Clase 2: El Proceso Generador de Datos (DGP)

Subtítulo: De la caja negra al dataset: medición, incertidumbre y aumento de datos
Institución: Facultad de Ciencias Empresariales y Economía — Universidad de Montevideo

1. Marco Teórico: El Proceso Generador de Datos (DGP)

1.1 ¿De dónde vienen los datos?

En la práctica profesional, solemos recibir un archivo `.csv` o una tabla SQL y asumimos que esa es la "realidad". Sin embargo, **los datos no existen en la naturaleza; se construyen mediante un acto de medición.**

Cualquier conjunto de datos observable es el resultado final de un **Proceso Generador de Datos** (DGP, por sus siglas en inglés: *Data Generating Process*). El DGP es el sistema o mecanismo subyacente (físico, biológico, social o económico) que produce los eventos que intentamos registrar.

1.2 Una caja negra

El estadístico Leo Breiman (2001) planteó que la naturaleza es una **caja negra** inaccesible en su totalidad. Dentro de la caja negra ocurren interacciones complejas, ruido e incertidumbre. Lo único que nosotros podemos observar son las salidas (Y) generadas a partir de ciertos insumos (X).

$$X \longrightarrow \boxed{\text{Caja Negra (DGP): } f(X) + \epsilon} \longrightarrow Y$$

¿Por qué un modelo "no sabe" cómo se generaron los datos?

> Un algoritmo de Machine Learning o un modelo estadístico de regresión solo procesa **matrices numéricas**. El modelo no tiene sentido común ni contexto ontológico: - No sabe si la variable `peso_g` se midió con una balanza digital o con una estimación a ojo. - No sabe si hubo un sesgo de selección en quién decidió responder la encuesta (Meng, 2018). - No comprende la relación causal subyacente; solo optimiza una función de pérdida sobre la muestra que se le entrega.
>
> Por eso, si el mecanismo de medición o el muestreo está roto, el modelo simplemente aprenderá y magnificará ese error (*"Garbage In, Garbage Out"*).

1.2 La Metáfora de la Caja Negra: Dinámica de Aula

Para vivenciar este concepto, realizaremos un experimento táctil en clase:

1. **El Objeto Oculto:** SE colocará una serie de objetos diversos dentro de una caja cerrada (la “Caja Negra” o DGP).
2. **La Medición Indirecta:** Cada estudiante tocará un objeto sin verlo durante 15 segundos.

Tipología de Variables a registrar:

- **Nominales:** Textura percibida (`suave`, `rugoso`, `poroso`).
- **Ordinales:** Nivel de rigidez (`blando`, `medio`, `rígido`).
- **Numéricas Discretas:** Cantidad de bordes o aristas detectadas ($k \in \mathbb{N}$).
- **Numéricas Continuas:** Estimación del peso en gramos ($\text{peso} \in \mathbb{R}^+$).
- **Metadatos:** Identificador del observador, tiempo de inspección y nivel de certeza asignado (1 al 5).

2. Construcción del Dataset en Código

A continuación, traducimos la experiencia táctil en una estructura de datos real en Python y R.

2.1 Crear dataframe en Python

En visual studio code o en python (PowerShell en Windows o Terminal en Mac):

pip install pandas numpy matplotlib

En RStudio:

```r
#install.packages(c("reticulate", "png", "jsonlite", "knitr", "rmarkdown"))
```

```r
reticulate::py_require(c("pandas", "numpy", "matplotlib"))
```

Diccionarios en python

```python
import pandas as pd
import numpy as np

# Definición del diccionaro crudo recabado en la dinámica
```

```
datos_caja = {
  "objeto_id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  "textura": ["suave", "rugoso", "poroso", "suave", "rugoso", "suave", "poroso", "rugoso", "suave", "poroso"],
  "rigidez": ["blando", "rígido", "medio", "blando", "rígido", "rígido", "blando", "medio", "medio", "blando"],
  "num_aristas": [0, 8, 0, 4, 12, 0, 0, 6, 4, 0],
    "peso_estimado_g": [45.0, 120.5, 15.0, 85.0, 210.0, 35.0, 18.5, 95.0, 70.0, 22.0],
    "certeza_observador": [5, 4, 3, 5, 4, 2, 4, 3, 4, 3]
}
```

```
# transformacion a dataframe
df_python = pd.DataFrame(datos_caja)

# Convertir tipos categóricos y ordinales
df_python["textura"] = df_python["textura"].astype("category")
df_python["rigidez"] = pd.Categorical(
    df_python["rigidez"],
    categories=["blando", "medio", "rígido"],
    ordered=True
)

print("Estructura del DataFrame en Python:")
```

```
Estructura del DataFrame en Python:
```

```
print(df_python.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   objeto_id           10 non-null     int64
 1   textura             10 non-null     category
 2   rigidez             10 non-null     category
 3   num_aristas         10 non-null     int64
 4   peso_estimado_g     10 non-null     float64
 5   certeza_observador  10 non-null     int64
dtypes: category(2), float64(1), int64(3)
memory usage: 628.0 bytes
None
```

```
print(df_python.head())
```

	objeto_id	textura	rigidez	num_aristas	peso_estimado_g	certeza_observador
0	1	suave	blando	0	45.0	5
1	2	rugoso	rígido	8	120.5	4
2	3	poroso	medio	0	15.0	3
3	4	suave	blando	4	85.0	5
4	5	rugoso	rígido	12	210.0	4

Datos simulados vs datos sintéticos

Datos simulados

Los datos simulados se generan mediante un proceso que implica imitar la operación, comportamiento o evolución de un sistema o proceso del mundo real a lo largo del tiempo, utilizando un modelo simplificado (matemático, físico o computacional).

Se ejecuta un modelo para imitar la dinámica de un sistema a lo largo del tiempo o bajo distintas condiciones (por ejemplo, simular una partida de ajedrez, el tráfico aéreo o la trayectoria de un huracán).

Define reglas, ecuaciones o mecánicas de comportamiento

3 Datos sintéticos

Crear datos sintéticos implica construir un ‘modelo’ de lo que creemos que hay dentro de la caja. Si nuestro modelo es simple, los datos sintéticos serán engañosos. Pero si lo hacemos bien, nos permite compartir datos para investigar sin mostrar el objeto real ni comprometer la privacidad de nadie.

Toma una muestra real de datos y usa algoritmos (Bootstrap, distribuciones ajustadas, redes GANs) para generar nuevas filas que se parezcan estadísticamente a la tabla original.

Métodos para crear datos sintéticos

Generación paramétrica

Generación Paramétrica (Distribución Normal): Para variables cuantitativas continuas (peso_estimado_g). Calculas la media (\(\mu\)) y desviación estándar (\(\sigma\)) de los 10 alumnos e “inventas” 500 pesos nuevos siguiendo esa campana de Gauss.

Re-muestreo

Re-muestreo Bootstrap:

El Bootstrap (Efron, 1979) asume que la muestra observada es la mejor aproximación disponible de la población desconocida.

```
graph LR
    subgraph Original ["Muestra (N=10)"]
        O1[Objeto 1]
        O2[Objeto 2]
        O3[Objeto 3]
    end

    subgraph Proceso ["Muestreo con Reemplazo"]
        P((Extraer + Devolver))
    end

    subgraph Bootstrap ["Bootstrap (N=100)"]
        B1[Objeto 1]
        B2[Objeto 1]
        B3[Objeto 3]
        B4[Objeto 2]
        B5[...]
    end

    Original --> Proceso
    Proceso --> Bootstrap
```

Datos sintéticos en python

```python
# ------------------------------------------------------------------------
# METODO 1: BOOTSTRAP (Muestreo con reemplazo para expandir a N = 100)
# ------------------------------------------------------------------------
np.random.seed(42)  # Semilla para reproducibilidad
df_bootstrap = df_python.sample(n=100, replace=True).reset_index(drop=True)

# ------------------------------------------------------------------------
# METODO 2: SINTETICO PARAMETRICO (Normal) para Peso (N = 500)
# ------------------------------------------------------------------------
# A. Calculamos los parámetros del modelo probabilístico (DGP estimado)
mu = df_python['peso_estimado_g'].mean()  # ~71.6 g
sigma = df_python['peso_estimado_g'].std()  # ~58.3 g

# B. Generamos 500 observaciones sintéticas
pesos_sinteticos = np.random.normal(loc=mu, scale=sigma, size=500)

# C. Aplicamos restricciones del mundo real (un peso no puede ser <= 0)
```

```
pesos_sinteticos = np.clip(pesos_sinteticos, a_min=1.0, a_max=None)

df_sintetico_pesos = pd.DataFrame(
    {'peso_estimado_g': pesos_sinteticos, 'tipo': 'Sintético'}
)
```

Limitaciones:

Reality gap

Representación limitada del mundo real: Los conjuntos de datos sintéticos pueden no capturar la complejidad y variabilidad completas de los datos reales.

Lectura recomendada: técnicas de aleatorización de dominios.

Tremblay, J., Prakash, A., Acuna, D., Brophy, M., Jampani, V., Anil, C., … & Birchfield, S. (2018, June). Training deep networks with synthetic data: Bridging the reality gap by domain randomization. In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW) (pp. 1082-10828). IEEE.

Kim, M., Liu, F., Jain, A., & Liu, X. (2023, June). Dcface: Synthetic face generation with dual condition diffusion model. In 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 12715-12725). IEEE.

Otras limitaciones

El peligro de los datos sintéticos: Generar datos sintéticos asumiendo una distribución Normal cuando el verdadero DGP es multimodal (varios objetos distintos) crea datos “fantasmas” que nunca existirían en la caja real.

Posible sesgo: El proceso de generación puede introducir sesgos si no refleja con precisión las características reales de la población.

Falta de contexto: Los conjuntos de datos sintéticos suelen carecer de la información contextual presente en los datos reales, lo que afecta su utilidad para el análisis. Generalización limitada: Los modelos entrenados con datos sintéticos pueden no tener un buen rendimiento con datos reales debido a diferencias en la distribución o los patrones subyacentes. Retos de validación: Puede resultar difícil validar modelos entrenados con datos sintéticos sin oportunidades de prueba en el mundo real.

Entonces ¿que es lo que hace crear un dataset de entrenamiento bueno?

- Alta Variación Inter-Clase (Cobertura del Dominio):
- Alta Variación Intra-Clase (Invariancia al Entorno):

- Mínimo Ruido en las Etiquetas (Fidelidad del Target):

Crear datos sintéticos o entrenar modelos complejos no compensa un dataset base con baja diversidad o etiquetas ruidosas. El límite de la capacidad de aprendizaje de un modelo no lo define la complejidad de su algoritmo, sino la riqueza estadística y la fidelidad del Proceso Generador de Datos (DGP) que se utilizó para entrenarlo.